

OBJECT-ORIENTED PROGRAMMING

All managed languages in the .NET Framework, such as Visual Basic and C#, provide full support for object-oriented programming including encapsulation, inheritance, and polymorphism.

Encapsulation means that a group of related properties, methods, and other members are treated as a single unit or object. Encapsulation is an Object Oriented Programming concept that binds together the data and functions that manipulate the data, and that keeps both safe from outside interference and misuse. Data encapsulation led to the important OOP concept of data hiding/Information Hiding.

Inheritance describes the ability to create new classes based on an existing class.

Polymorphism means that you can have multiple classes that can be used interchangeably, even though each class implements the same properties or methods in different ways.

Classes and Objects

The terms *class* and *object* are sometimes used interchangeably, but in fact, classes describe the *type* of objects, while objects are usable *instances* of classes. So, the act of creating an object is called *instantiation*. Using the blueprint analogy, a class is a blueprint, and an object is a building made from that blueprint.

```
class Child
{
    private int age;
    private string name;
    // Default constructor:
    public Child()
    {
        name = "N/A";
    }
    // Constructor:
    public Child(string name, int age)
    {
        this.name = name;
        this.age = age;
    }
}
```

```

// Printing method:
public void PrintChild()
{
    Console.WriteLine("{0}, {1} years old.", name, age);
}
}

```

Class Members

Each class can have different *class members* that include properties that describe class data, methods that define class behavior, and events that provide communication between different classes and objects.

Properties and Fields

Fields and properties represent information that an object contains. Fields are like variables because they can be read or set directly.

```

Class SampleClass
{
    public string sampleField;
}

```

Methods

A *method* is an action that an object can perform. A method is a code block that contains a series of statements. A program causes the statements to be executed by calling the method and specifying any required method arguments. In C#, every executed instruction is performed in the context of a method.

Methods are declared in a class or struct by specifying the access level such as **public** or **private**, optional modifiers such as **abstract** or **sealed**, the return value, the name of the method, and any method parameters. These parts together are the signature of the method. Methods are declared in a class or struct by specifying the access level such as **public** or **private**, optional modifiers such as **abstract** or **sealed**, the return value, the name of the method, and any method parameters. These parts together are the signature of the method.

```

class SampleClass
{
    public int sampleMethod(string sampleParam)
    {
        // Insert code here
    }
}

```

Method Access

Calling a method on an object is like accessing a field. After the object name, add a period, the name of the method, and parentheses. Arguments are listed within the parentheses, and are separated by commas. The methods of the `Motorcycle` class can therefore be called as in the following example:

```

class TestMotorcycle : Motorcycle
{
    public override double GetTopSpeed()
    {
        return 108.4;
    }

    static void Main()
    {
        TestMotorcycle moto = new TestMotorcycle();
        moto.StartEngine();
        moto.AddGas(15);
        moto.Drive(5, 20);
        double speed = moto.GetTopSpeed();
        Console.WriteLine("My top speed is {0}", speed);
    }
}

```

Method Parameters vs. Arguments

The method definition specifies the names and types of any parameters that are required. When calling code calls the method, it provides concrete values called arguments for each parameter.

The arguments must be compatible with the parameter type but the argument name (if any) used in the calling code does not have to be the same as the parameter named defined in the method. For example:

```
public void Caller()
{
    int numA = 4;
    // Call with an int variable.
    int productA = Square(numA);
    int numB = 32;
    // Call with another int variable.
    int productB = Square(numB);
    // Call with an integer literal.
    int productC = Square(12);
    // Call with an expression that evaluates to int.
    productC = Square(productA * 3);
}

int Square(int i)
{
    // Store input argument in a local variable.
    int input = i;
    return input * input;
}
```

Return Values

Methods can return a value to the caller. If the return type, the type listed before the method name, is not **void**, the method can return the value by using the **return** keyword. A statement with the **return** keyword followed by a value that matches the return type will return that value to the method caller. The **return** keyword also stops the execution of the method. If the return type is **void**, a **return** statement without a value is still useful to stop the execution of the method. Without the **return** keyword, the method will stop executing when it reaches the end of the code block. Methods with a non-void return type are required to use the **return** keyword to return a value. For example, these two methods use the **return** keyword to return integers:

```

class SimpleMath
{
    public int AddTwoNumbers(int number1, int number2)
    {
        return number1 + number2;
    }

    public int SquareANumber(int number)
    {
        return number * number;
    }
}

```

To use a value returned from a method, the calling method can use the method call itself anywhere a value of the same type would be sufficient. You can also assign the return value to a variable.

```

int result = obj.AddTwoNumbers(1, 2);
result = obj.SquareANumber(result);
// The result is 9.
Console.WriteLine(result);

```

Constructors

Constructors are kind of like class methods that are executed automatically when an class object is created. Constructors usually initialize the data members of the new object. A constructor can run only once when a class instance/Object is created. Furthermore, the code in the constructor always runs before any other code in a class. However, you can create multiple constructor overloads in the same way as for any other method. Constructors do not return any kind of values.

To define a constructor for a class:

```

public class SampleClass
{
    int i;
    public SampleClass()
    { random r = new random();
        i = r.Next();
    }
}

```

Nested Classes

A class defined within another class is called *nested*. By default, the nested class is private.

```
class Container
{
    class Nested
    {
        // Add code here.
    }
}
```

To create an instance of the nested class, use the name of the container class followed by the dot and then followed by the name of the nested class:

```
Container.Nested nestedInstance = new Container.Nested();
```

Access Modifiers and Access Levels

All classes and class members can specify what access level they provide to other classes by using *access modifiers*. The following access modifiers are available:

C# Modifier	Definition
public	The type or member can be accessed by any other code in any assembly that references it.
private	The type or member can only be accessed by code in the same class.
protected	The type or member can only be accessed by code in the same class or in a derived class.

Instantiating Classes

To create an object, you need to instantiate a class, or create a class instance.

```
SampleClass sampleObject = new SampleClass();
```

After instantiating a class, you can assign values to the instance's properties and fields and invoke class methods.

```
// Set a property value.  
sampleObject.sampleProperty = "Sample String";  
// Call a method.  
sampleObject.sampleMethod();
```

To assign values to properties during the class instantiation process, use object initializers:

```
// Set a property value.  
SampleClass sampleObject = new SampleClass  
    { FirstProperty = "A", SecondProperty = "B" };
```

Static (Shared) Classes and Members

A static (shared in Visual Basic) member of the class is a property, procedure, or field that is shared by all instances of a class. Static (shared) classes in C# and modules in Visual Basic have static (shared) members only and cannot be instantiated. Static (shared) members also cannot access non-static (non-shared) properties, fields or methods.

To define a static (shared) member:

```
static class SampleClass  
{  
    public static string SampleString = "Sample String";  
}
```

To access static (shared) member, use the name of the class without creating an object of this class:
`Console.WriteLine(SampleClass.SampleString);`

Anonymous Types

Anonymous types enable you to create objects without writing a class definition for the data type. Instead, the compiler generates a class for you. The class has no usable name and contains the properties you specify in declaring the object. The point to remember is that anything declared with `var` needs to be assigned a value to it at the of declaration.

To create an instance of an anonymous type:

```
// sampleObject is an instance of a simple anonymous type.  
var sampleObject =  
    new { FirstProperty = "A", SecondProperty = "B" };
```

Inheritance

Inheritance enables you to create a new class that reuses, extends, and modifies the behavior that is defined in another class. The class whose members are inherited is called the *base class*, and the class that inherits those members is called the *derived class*. However, all classes in both C# and Visual Basic implicitly inherit from the [Object](#) class that supports .NET class hierarchy and provides low-level services to all classes.

To inherit from a base class:

```
class DerivedClass:BaseClass{}
```

By default all classes can be inherited. However, you can specify whether a class must not be used as a base class, or create a class that can be used as a base class only.

To specify that a class cannot be used as a base class:

```
public sealed class A { }
```

To specify that a class can be used as a base class only and cannot be instantiated:

```
public abstract class B { }
```

Overriding Members

By default, a derived class inherits all members from its base class. If you want to change the behavior of the inherited member, you need to override it. That is, you can define a new implementation of the method, property or event in the derived class.

The following modifiers are used to control how properties and methods are overridden:

C# Modifier	Definition
virtual (C# Reference)	Allows a class member to be overridden in a derived class.
override (C# Reference)	Overrides a virtual (overridable) member defined in the base class.
abstract (C# Reference)	Requires that a class member to be overridden in the derived class.

Interfaces

Interfaces, like classes, define a set of properties, methods, and events. But unlike classes, interfaces do not provide implementation. They are implemented by classes, and defined as separate entities from classes. An interface represents a contract, in that a class that implements an interface must implement every aspect of that interface exactly as it is defined.

```
interface ISampleInterface
```

```
{  
    void doSomething();  
}
```

To implement an interface in a class:

```
class SampleClass : ISampleInterface
```

```
{  
    void ISampleInterface.doSomething()  
    {  
        // Method implementation.  
    }  
}
```

Polymorphism Overview

Polymorphism is often referred to as the third pillar of object-oriented programming, after encapsulation and inheritance. Polymorphism is a Greek word that means "many-shaped" and it has two distinct aspects:

- At run time, objects of a derived class may be treated as objects of a base class in places such as method parameters and collections or arrays. When this occurs, the object's declared type is no longer identical to its run-time type.
- Base classes may define and implement virtual *methods*, and derived classes can override them, which means they provide their own definition and implementation. At run-time, when client code calls the method, the CLR looks up the run-time type of the object and invokes that override of the virtual method. Thus, in your source code you can

call a method on a base class and cause a derived class's version of the method to be executed.

Virtual methods enable you to work with groups of related objects in a uniform way. For example, suppose you have a drawing application that enables a user to create various kinds of shapes on a drawing surface. You do not know at compile time which specific types of shapes the user will create. However, the application has to keep track of all the various types of shapes that are created, and it has to update them in response to user mouse actions. You can use polymorphism to solve this problem in two basic steps:

1. Create a class hierarchy in which each specific shape class derives from a common base class.
2. Use a virtual method to invoke the appropriate method on any derived class through a single call to the base class method.

First, create a base class called Shape, and derived classes such as Rectangle, Circle, and Triangle. Give the Shape class a virtual method called Draw, and override it in each derived class to draw the particular shape that the class represents. Create a List<Shape> object and add a Circle, Triangle and Rectangle to it. To update the drawing surface, use a foreach loop to iterate through the list and call the Draw method on each Shapeobject in the list. Even though each object in the list has a declared type of Shape, it is the run-time type (the overridden version of the method in each derived class) that will be invoked.

```
public class Shape
{
    // A few example members
    public int X { get; private set; }
    public int Y { get; private set; }
    public int Height { get; set; }
    public int Width { get; set; }

    // Virtual method
    public virtual void Draw()
    {
        Console.WriteLine("Performing base class drawing tasks");
    }
}
```

```

class Circle : Shape
{
    public override void Draw()
    { // Code to draw a circle...
        Console.WriteLine("Drawing a circle");
        base.Draw();
    }
}

class Rectangle : Shape
{
    public override void Draw()
    { // Code to draw a rectangle...
        Console.WriteLine("Drawing a rectangle");
        base.Draw();
    }
}

class Triangle : Shape
{ public override void Draw()
    { // Code to draw a triangle...
        Console.WriteLine("Drawing a triangle");
        base.Draw();
    }
}

#include System.Collections.Generic;

class Program
{ static void Main(string[] args)
    {
        // Polymorphism at work #1: a Rectangle, Triangle and Circle
        // can all be used where ever a Shape is expected. No cast is
        // required because an implicit conversion exists from a derived
        // class to its base class.
        List<Shape> shapes = new List<Shape>();
        shapes.Add(new Rectangle());
        shapes.Add(new Triangle());
        shapes.Add(new Circle());
        // Polymorphism at work #2: the virtual method Draw is
        // invoked on each of the derived classes, not the base class.
    }
}

```

```

        foreach (Shape s in shapes)
        {
            s.Draw();
        }
        // Keep the console open in debug mode.
        Console.WriteLine("Press any key to exit.");
        Console.ReadKey();
    }
}
/* Output:
    Drawing a rectangle
    Performing base class drawing tasks
    Drawing a triangle
    Performing base class drawing tasks
    Drawing a circle
    Performing base class drawing tasks    */

```

Virtual Members

When a derived class inherits from a base class, it gains all the methods, fields, properties and events of the base class. The designer of the derived class can choose whether to

- override virtual members in the base class,
- inherit the closest base class method without overriding it
- define new non-virtual implementation of those members that hide the base class implementations

A derived class can override a base class member only if the base class member is declared as virtual or abstract. The derived member must use the override keyword to explicitly indicate that the method is intended to participate in virtual invocation. The following code provides an example.

```

public class BaseClass
{
    public virtual void DoWork() { }
    public virtual int WorkProperty
    {
        get { return 0; }
    }
}

```

```

public class DerivedClass : BaseClass
{
    public override void DoWork() { }
    public override int WorkProperty
    {
        get { return 0; }
    }
}

```

Fields cannot be virtual; only methods, properties, events and indexers can be virtual.

Preventing Derived Classes from Overriding Virtual Members

Virtual members remain virtual indefinitely, regardless of how many classes have been declared between the virtual member and the class that originally declared it. If class A declares a virtual member, and class B derives from A, and class C derives from B, class C inherits the virtual member, and has the option to override it, regardless of whether class B declared an override for that member. The following code provides an example:

```

public class A
{
    public virtual void DoWork() { }
}
public class B : A
{
    public override void DoWork() { }
}

```

A derived class can stop virtual inheritance by declaring an override as **sealed**. This requires putting the **sealed** keyword before the **override** keyword in the class member declaration. The following code provides an example:

```

public class C : B
{
    public sealed override void DoWork() { }
}

```

Accessing Base Class Virtual Members from Derived Classes

A derived class that has replaced or overridden a method or property can still access the method or property on the base class using the base keyword. The following code provides an example:

```
public class Base
{
    public virtual void DoWork() { /*...*/ }
}
public class Derived : Base
{
    public override void DoWork()
    {
        //Perform Derived's work here
        //...
        // Call DoWork on base class
        base.DoWork();
    }
}
```

Encapsulation

Encapsulation is defined as the process of enclosing one or more items within a physical or logical package. Encapsulation, in object-oriented programming methodology, prevents access to implementation details. Abstraction and encapsulation are related features in object-oriented programming. Abstraction allows making relevant information visible and encapsulation enables a programmer to *implement the desired level of abstraction*. Encapsulation is implemented by using **access specifiers**. An **access specifier** defines the scope and visibility of a class member. C# supports the following access specifiers:

- Public
- Private
- Protected
- Internal
- Protected internal

Encapsulation in C# is an OOP concept that binds the data and methods together to create a single unit. Encapsulation is defined as the wrapping up of data under a single unit. As defined it is the mechanism that binds together code and the data it manipulates creating a protective layer that prevents the data from being accessed by unauthorized code.

- Technically in encapsulation, the variables or data of a class are hidden from any other classed and can be accessed only through any member function of own class in which they are declared.
- In encapsulation, the data in a class is hidden from other classes, so it is also known as data-hiding

Encapsulation is implemented by using access specifiers. An access specifier defines the scope and visibility of a class member. C# supports the following access specifiers

1. Public

Public access specifier allows a class to expose its member variables and functions to other classes and objects. There are no restrictions on accessing public members

```
class SampleClass
{
    public int x; // No access restrictions.
}
```

Example

```
class PointTest
{
    public int x;
    public int y;
}

class MainClass4 {
    public static void Main() {
        var p = new PointTest();
        // Direct access to public members.
        p.x = 10;
        p.y = 15;
        Console.WriteLine($"x = {p.x}, y = {p.y}");
    } }
```

2. Private

Private access specifier allows a class to hide its member variables and member functions from other objects. Only members of the same class can access its private members. Even an instance of a class cannot access its private members. Private access is the least permissive access level. Private members are accessible only within the body of the class or the struct in which they are declared. If no access modifier is defined it is private by default

```
class Employee
{
    private int i;
    double d; // private access by default
}
```

3. Protected

Protected access specifier allows a child class to access the member variables and member functions of its base class. This way it helps in implementing inheritance.

```
class A
{
    protected int x = 123;
}

class B : A
{
    static void Main()
    {
        var a = new A();
        var b = new B();
        // a.x = 10;
        // Error, because x can only be accessed by classes derived from A. in
        // this case only class B
        // OK, because this class derives from A.
        b.x = 10;
    }
}
```


4. Internal

Internal access specifier allows a class to expose its member variables and member functions to other functions and objects in the current assembly. In other words, any member with internal access specifier can be accessed from any class or method defined within the application in which the member is defined.

5. Protected internal

The protected internal access specifier allows a class to hide its member variables and member functions from other class objects and functions, except a child class within the same application. The protected internal keyword combination is a member access modifier. A protected internal member is accessible from the current assembly or from types that are derived from the containing class.

6. private protected

The private protected keyword combination is a member access modifier. A private protected member is accessible by types derived from the containing class, but only within its containing assembly

Declared accessibility

The ***declared accessibility*** of a member can be one of the following:

- Public, which is selected by including a public modifier in the member declaration. The intuitive meaning of public is "access not limited".
- Protected, which is selected by including a protected modifier in the member declaration. The intuitive meaning of protected is "access limited to the containing class or types derived from the containing class".
- Internal, which is selected by including an internal modifier in the member declaration. The intuitive meaning of internal is "access limited to this program".
- Protected internal (meaning protected or internal), which is selected by including both a protected and an internal modifier in the member declaration. The intuitive meaning of protected internal is "access limited to this program or types derived from the containing class".
- Private, which is selected by including a private modifier in the member declaration. The intuitive meaning of private is "access limited to the containing type".

Depending on the context in which a member declaration takes place, only certain types of declared accessibility are permitted. Furthermore, when a member declaration does not include any access modifiers, the context in which the declaration takes place determines the default accessibility.

Implicit behaviors of accessibility

- Namespaces implicitly have public declared accessibility. No access modifiers are allowed on namespace declarations.
- Class members can have any of the five kinds of declared accessibility and default to private declared accessibility. (Note that a type declared as a member of a class can have any of the five kinds of declared accessibility, whereas a type declared as a member of a namespace can have only public or internal declared accessibility.)
- Struct members can have public, internal, or private declared accessibility and default to private declared accessibility because structs are implicitly sealed.
- Interface members implicitly have public declared accessibility. No access modifiers are allowed on interface member declarations.
- Enumeration members implicitly have public declared accessibility. No access modifiers are allowed on enumeration member declarations.

Abstract Classes and Class Members

Classes can be declared as abstract by putting the keyword `abstract` before the class definition. For example:

```
public abstract class A
{
    // Class members here.
}
```

An abstract class cannot be instantiated. The purpose of an abstract class is to provide a common definition of a base class that multiple derived classes can share. For example, a class library may define an abstract class that is used as a parameter to many of its functions, and require programmers using that library to provide their own implementation of the class by creating a derived class.

Abstract classes may also define abstract methods. This is accomplished by adding the keyword `abstract` before the return type of the method. For example:

```
public abstract class D
{
    public abstract void DoWork(int i);
}
```

Abstract methods have no implementation, so the method definition is followed by a semicolon instead of a normal method block. Derived classes of the abstract class must implement all abstract methods. When an abstract class inherits a virtual method from a base class, the abstract class can override the virtual method with an abstract method. For example:

```
public abstract class E : D
{
    // abstract method declared in abstract class.
    public abstract override void DoWork(int i);
}

public class F : E
{
    public override void DoWork(int i)
    {
        // Proper method code goes here.
    }
}
```

If a virtual method is declared abstract, it is still virtual to any class inheriting from the abstract class. A class inheriting an abstract method cannot access the original implementation of the method—in the previous example, `DoWork` on class `F` cannot call `DoWork` on class `D`. In this way, an abstract class can force derived classes to provide new method implementations for virtual methods.

Sealed Classes and Class Members

Classes can be declared as sealed by putting the keyword sealed before the class definition. For example:

```
public sealed class D
{
    // Class members here.
}
```

A sealed class cannot be used as a base class. For this reason, it cannot also be an abstract class. Sealed classes prevent derivation. Because they can never be used as a base class, some run-time optimizations can make calling sealed class members slightly faster.

A method, indexer, property, or event, on a derived class that is overriding a virtual member of the base class can declare that member as sealed. This negates the virtual aspect of the member for any further derived class. This is accomplished by putting the sealed keyword before the override keyword in the class member declaration. For example:

```
public class D : C
{
    public sealed override void DoWork() { }
}
```

Exception Handling

An exception is a problem that arises during the execution of a program. A C# exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero. Exceptions provide a way to transfer control from one part of a program to another. C# exception handling is built upon four keywords: try, catch, finally, and throw.

- try: A try block identifies a block of code for which particular exceptions is activated. It is followed by one or more catch blocks.
- catch: A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The catch keyword indicates the catching of an exception.
- finally: The finally block is used to execute a given set of statements, whether an exception is thrown or not thrown. For example, if you open a file, it must be closed whether an exception is raised or not.

Syntax

Assuming a block raises an exception, a method catches an exception using a combination of the try and catch keywords. A try/catch block is placed around the code that might generate an exception. Code within a try/catch block is referred to as protected code, and the syntax for using try/catch looks like the following:

```
try
{
    // statements causing exception
}
catch( ExceptionName e1 )
{
    // error handling code
}
catch( ExceptionName e2 )
{
    // error handling code
}
catch( ExceptionName eN )
{
    // error handling code
}
finally
{
    // statements to be executed
}
```

You can list down multiple catch statements to catch different type of exceptions in case your try block raises more than one exception in different situations.

Exception Classes in C#

C# exceptions are represented by classes. The exception classes in C# are mainly directly or indirectly derived from the System.Exception class. Some of the exception classes derived from the System.Exception class are the System.ApplicationException and System.SystemException classes.

The System.ApplicationException class supports exceptions generated by application programs. Hence the exceptions defined by the programmers should derive from this class.

The System.SystemException class is the base class for all predefined system exception.

The following table provides some of the predefined exception classes derived from the Sytem.SystemException class:

Exception Class	Description
System.IO.IOException	Handles I/O errors.
System.IndexOutOfRangeException	Handles errors generated when a method refers to an array index out of range.
System.ArrayTypeMismatchException	Handles errors generated when type is mismatched with the array type.
System.NullReferenceException	Handles errors generated from deferencing a null object.
System.DivideByZeroException	Handles errors generated from dividing a dividend with zero.
System.InvalidCastException	Handles errors generated during typecasting.
System.OutOfMemoryException	Handles errors generated from insufficient free memory.
System.StackOverflowException	Handles errors generated from stack overflow.

Handling Exceptions

C# provides a structured solution to the exception handling in the form of try and catch blocks. Using these blocks the core program statements are separated from the error-handling statements.

These error handling blocks are implemented using the try, catch, and finally keywords. Following is an example of throwing an exception when dividing by zero condition occurs:

```
using System;
namespace ErrorHandlingApplication
{
    class DivNumbers
    {
        int result;
        DivNumbers()
        {
            result = 0;
        }
        public void division(int num1, int num2)
        {
            try
            {
                result = num1 / num2;
            }
            catch (DivideByZeroException e)
            {
                Console.WriteLine("Exception caught: {0}", e);
            }
            finally
            {
                Console.WriteLine("Result: {0}", result);
            }
        }
        static void Main(string[] args)
        {
            DivNumbers d = new DivNumbers();
            d.division(25, 0);
            Console.ReadKey();
        }
    }
}
```

When the above code is compiled and executed, it produces the following result:

Exception caught: System.DivideByZeroException: Attempted to divide by zero.

at ...

Result: 0

C# - Structures

In C#, a structure is a value type data type. It helps you to make a single variable hold related data of various data types. The **struct** keyword is used for creating a structure.

Defining a Structure

To define a structure, you must use the struct statement. The struct statement defines a new data type, with more than one member for your program.

Features of C# Structures.

- Structures can have methods, fields, indexers, properties, operator methods, and events.
- Structures can have defined constructors, but not destructors. However, you cannot define a default constructor for a structure. The default constructor is automatically defined and can't be changed.
- Unlike classes, structures cannot inherit other structures or classes.
- Structures cannot be used as a base for other structures or classes.
- A structure can implement one or more interfaces.
- Structure members cannot be specified as abstract, virtual, or protected.
- When you create a struct object using the **New** operator, it gets created and the appropriate constructor is called.
- Unlike classes, structs can be instantiated without using the New operator.
- If the New operator is not used, the fields will remain unassigned and the object cannot be used until all the fields are initialized.

Class vs Structure

Classes and Structures have the following basic differences:

- classes are reference types and structs are value types
- structures do not support inheritance
- structures cannot have default constructor let us rewrite the previous example:

Example program written below

```
using System;
struct Books
{
    public string title;
    public string author;
    public string subject;
    public int book_id;
    // parameterized constructors can be defined only
    // default parameter less constructors are included implicitly but cannot be
    // coded in constructor e.g. this is not allowed
    public Books(string title, string auth, string sub, int id)
    {
        this.title = title;
        author = auth;
        subject = sub;
        book_id = id;
        // keyword "this" is used to refer to global variable/ fields
        // in cases where there are local variable of a method having same name
    }
    public void setValues(string t, string a, string s, int id)
    {
        title = t;
        author = a;
        subject = s;
        book_id = id;
    }
}
```

```

    public void display()
    {
        Console.WriteLine("Title : " + title);
        Console.WriteLine("Author : " + author);
        Console.WriteLine("Subject : " + subject);
        Console.WriteLine("Book_id : " + book_id);
    }
}
public class testStructure
{
    public static void Main(string[] args)
    {
        /* Declare Book1 of type Book */
        Books Book1 = new Books("C Programming", "Nuha Ali", "C Programming Tutorial",
6495407);
        /* Declare Book2 of type Book */
        Books Book2;
        Book2.title = "Telecom Billing";
        Book2.author = "Zara Ali";
        Book2.subject = "Telecom Billing Tutorial";
        Book2.book_id = 6495700;
        /* print Book1 info */
        Book1.display();
        /* print Book2 info */
        Book2.display();
        Console.ReadKey();
    }
}

```

Output

```

Title : C Programming
Author : Nuha Ali
Subject : C Programming Tutorial
Book_id : 6495407

```

```

Title : Telecom Billing
Author : Zara Ali

```

Subject : Telecom Billing Tutorial

Book_id : 6495700

Generics

Generics allow you to delay the specification of the data type of programming elements in a class or a method, until it is actually used in the program. In other words, generics allow you to write a class or method that can work with any data type.

You write the specifications for the class or the method, with substitute parameters for data types. When the compiler encounters a constructor for the class or a function call for the method, it generates code to handle the specific data type. A simple example would help understanding the concept:

```
using System;
using System.Collections.Generic;
namespace GenericApplication
{
    public class MyGenericArray<T>
    {
        private T[] array;
        public MyGenericArray(int size)
        {
            array = new T[size + 1];
        }
        public T getItem(int index)
        {
            return array[index];
        }
        public void setItem(int index, T value)
        {
            array[index] = value;
        }
    }
}
```

```

class Tester
{
    static void Main(string[] args)
    {
        //declaring an int array
        MyGenericArray<int> intArray = new MyGenericArray<int>(5);
        //setting values
        for (int c = 0; c < 5; c++)
        {
            intArray.setItem(c, c * 5);
        }
        //retrieving the values
        for (int c = 0; c < 5; c++)
        {
            Console.Write(intArray.getItem(c) + " ");
        }
        Console.WriteLine();
        //declaring a character array
        MyGenericArray<char> charArray = new MyGenericArray<char>(5);
        //setting values
        for (int c = 0; c < 5; c++)
        {
            //((char) converts the given integer to its corresponding ASCII character
            charArray.setItem(c, (char)(c + 97));
        }
        //retrieving the values
        for (int c = 0; c < 5; c++)
        {
            Console.Write(charArray.getItem(c) + " ");
        }
        Console.WriteLine();
        Console.ReadKey();
    }
}

```

When the above code is compiled and executed, it produces the following result:

0 5 10 15 20

a b c d e

Generic Methods

In the previous example, we have used a generic class; we can declare a generic method with a type parameter. The following program illustrates the concept: **Example**

```
using System;
using System.Collections.Generic;
namespace GenericMethodAppl
{
    class Program
    {
        static void Swap<T>(ref T lhs, ref T rhs)
        {
            T temp; temp = lhs; lhs = rhs; rhs = temp;
        }
        static void Main(string[] args)
        {
            int a, b; char c, d; a = 10; b = 20; c = 'I'; d = 'V';
            //display values before swap:
            Console.WriteLine("Int values before calling swap:");
            Console.WriteLine("a = {0}, b = {1}", a, b);
            Console.WriteLine("Char values before calling swap:");
            Console.WriteLine("c = {0}, d = {1}", c, d);
            //call swap
            Swap<int>(ref a, ref b); Swap<char>(ref c, ref d);
            //display values after swap:
            Console.WriteLine("Int values after calling swap:");
            Console.WriteLine("a = {0}, b = {1}", a, b);
            Console.WriteLine("Char values after calling swap:");
            Console.WriteLine("c = {0}, d = {1}", c, d);
            Console.ReadKey();
        }
    }
}
```

When the above code is compiled and executed, it produces the following result: Int values before calling swap:

a = 10, b = 20

Char values before calling swap:

c = I, d = V

Int values after calling swap:

a = 20, b = 10

Char values after calling swap:

c = V, d = I

Features of Generics

Using generics is a technique that enriches your programs in the following ways:

- It helps you to maximize code reuse, type safety, and performance.
- You can create generic collection classes. The .NET Framework class library contains several new generic collection classes in the *System.Collections.Generic* namespace.
- You may use these generic collection classes instead of the collection classes in the *System.Collections* namespace.
- You can create your own generic interfaces, classes, methods, events and delegates.
- You may create generic classes constrained to enable access to methods on particular data types.
- You may get information on the types used in a generic data type at run-time by means of reflection.

Object Class

Supports all classes in the .NET class hierarchy and provides low-level services to derived classes. This is the ultimate base class of all .NET classes. Any type in C# can be assigned to object class.

```
Object a = 12;
```

```
a = "Hello";
```